



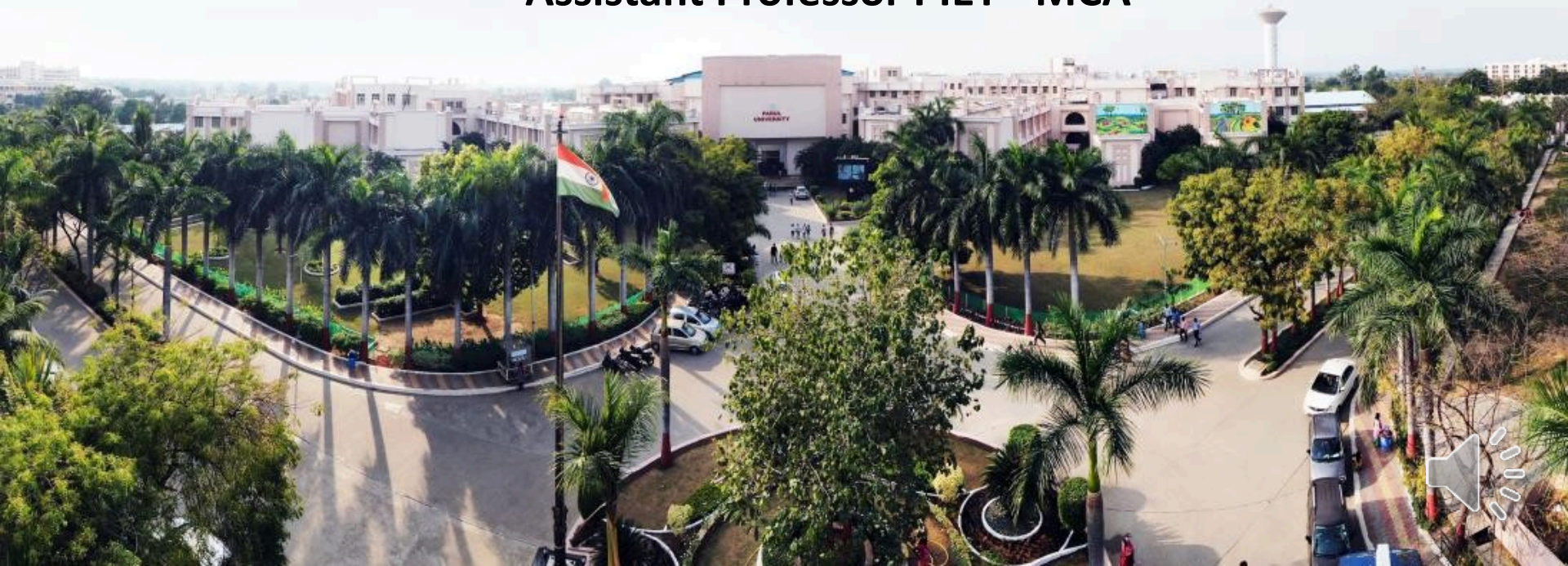
05201302

Web Application Development

MCA MSc-IT Semester 3

Prof Faruk Abdulla

Assistant Professor PIET - MCA





Unit-3

ASP.NET Validation Controls, State Management & Navigation Validation Control: Controls, Basic validation controls, Validation techniques, Using advanced validation controls. State Management: Using view state, Using session state, Using application state, Using cookies and URL encoding. Master Pages: Creating master pages, Content pages, nesting master pages, Accessing master page controls from a content page. Themes : Skin File and CSS Navigation: Site navigation, Using site navigation controls.



Why ASP.NET?

This unit introduces the foundational components that power modern ASP.NET web development. It covers:

- **Validation Controls**
- **State Management** (Client & Server side)
- **Master Pages**
- **Themes and CSS Styling**
- **Site Navigation Controls**



ASP .NET

*“ASP .NET is Used to Building Dynamic,
Interactive, and Scalable Web Applications”*



Why ASP.NET?

Rapid Development: Rich server controls & drag-drop support.

Separation of Concerns: Code-behind files for logic.

Stateful Web Experience: Built-in state management options.

Security & Validation: Prebuilt validation mechanisms.

Real-World Use: Portals, e-commerce, enterprise solutions.

Validation Controls

ASP.NET validation controls ensure that **user input meets specific criteria** before processing (e.g., required fields, numeric ranges).

Common Validation Controls

RequiredFieldValidator: Ensures a field is not empty.

RangeValidator: Checks if value is within a range.

CompareValidator: Compares input with another field/value.

RegularExpressionValidator: Validates pattern (e.g., email format).

CustomValidator: Custom logic (written in C#).

ValidationSummary: Displays all validation errors in one place.

Client-Side vs Server-Side Validation

Client-Side: Runs in the browser (faster, less secure).

Server-Side: Runs on the server (secure, ensures validation even if JS is disabled).

Client-Side Validation Example

Runs in the **browser** using built-in ASP.NET scripts (fast feedback).

```
<asp:TextBox ID="txtName" runat="server" />  
<asp:RequiredFieldValidator ID="rfvName" runat="server"  
    ControlToValidate="txtName" ErrorMessage="Name is required!"  
    EnableClientScript="true" />
```

Shows an error instantly if the field is empty (no server request).

Server-Side Validation

Runs on the **server** after form submission (secure & reliable).

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtName.Text))
    {
        lblMessage.Text = "Name is required!";
    }
}
```

Even if JavaScript is disabled, the server revalidates before processing.



What is Input Validation?

Validation is the process of checking whether user input meets required criteria before processing.

Why Validate?

- Prevent invalid data
- Improve user experience with clear feedback
- Protect server-side logic and database integrity

Example: Ensuring the 'Email' field has a valid format in a registration form.

RequiredFieldValidator

- Ensures that a field is not left empty. Commonly used for mandatory fields like Name, Email, or Password.

```
<asp:RequiredFieldValidator ID="rfvName" runat="server"  
    ControlToValidate="txtName" ErrorMessage="Name is required!" /  
>
```



RangeValidator

- ❑ Checks if input falls within a range.
- ❑ Ensures a value entered by the user is within a specific range.
- ❑ Works for numeric, date, or string values.

Key Properties:

- **ControlToValidate**: ID of the control to validate.
- **MinimumValue** & **MaximumValue**: Range limits.
- **Type**: Data type (Integer, Double, Date, String).
- **ErrorMessage**: Message to display when validation fails.

```
<asp:RangeValidator ControlToValidate="txtAge"  
MinimumValue="18" MaximumValue="60" Type="Integer"  
ErrorMessage="Age must be between 18–60" runat="server" />
```



CompareValidator

- ❑ Compares two values (e.g., password & confirm password).
- ❑ Validates format using regex.
- ❑ Example: Validate email address.

Validates format using regex.

Example: Validate email address, age Limit etc

```
<asp:RangeValidator ControlToValidate="txtAge"  
MinimumValue="18" MaximumValue="60" Type="Integer"  
ErrorMessage="Age must be between 18–60" runat="server" />
```




CompareValidator

□ CompareValidator – Comparing Two Inputs

Password confirmation (Password == Confirm Password).

Date comparison (StartDate < EndDate).

Data type validation (e.g., ensuring numeric entry).

```
<asp:RangeValidator ControlToValidate="txtAge"  
MinimumValue="18" MaximumValue="60" Type="Integer"  
ErrorMessage="Age must be between 18–60" runat="server" />
```

RegularExpressionValidator

Validates input against a **regular expression pattern**, such as email, phone number, or postal code formats.

```
<asp:RegularExpressionValidator ID="revEmail" runat="server"  
    ControlToValidate="txtEmail" ValidationExpression="\w+@\w+\.\w+"  
    ErrorMessage="Enter a valid email address!" />
```

CustomValidator

Provides **custom validation logic** in code-behind. Useful for scenarios like database lookups (e.g., checking if a username already exists).

```
protected void cvUser_ServerValidate(object source,  
ServerValidateEventArgs args)  
{  
    args.IsValid = !IsUsernameTaken(args.Value); // Custom logic  
}
```

ValidationSummary

Displays all validation errors in a **single place**, making it easier for users to see what needs fixing.

```
<asp:ValidationSummary ID="vsErrors" runat="server"  
    HeaderText="Please fix the following errors:" />
```



State management

State management is the technique of **storing and maintaining data** (the "state") of a user **between multiple requests/pages** in a web application.

When you browse a website, each page load is **independent** — the server does **not remember** who you are or what you did before. This is because **HTTP is stateless**.

State management

HTTP is stateless: Each request is independent – the server doesn't remember anything about the user.

What happens without state?

User logs in → Clicks a new page → Login information is lost.

Shopping cart items disappear when navigating to a new page.



The Need for State Management:

- **Preserve user data across multiple requests/pages.**
- **Provide personalized experiences:**
 - Keep user logged in.
 - Maintain shopping cart/session data.
 - Store preferences (themes, language).
- **Improve usability & continuity:** Avoid re-entering information.
- **Support multi-step workflows:** Forms, checkout processes, surveys.



The Need for State Management:

- **E-commerce:** Cart items remain even when browsing different products.
- **Banking portals:** Maintain authentication during transactions.
- **Education platforms:** Track quiz progress or lessons completed.

ViewState

- A mechanism in ASP.NET to **persist page and control values across postbacks**.
- Stored as a **hidden field** (`__VIEWSTATE`) in the page.
- **Base64 encoded** (not encrypted by default).
- **Automatic for server controls:** TextBox, DropDownList, etc.

ViewState

- User submits form → ASP.NET **serializes control states** into `__VIEWSTATE`.
- Data is sent to the **browser** in a hidden field.
- On postback → The **ViewState is sent back to the server** → ASP.NET reconstructs controls.



ViewState: Advantages & Limitations

Advantages:

Simple & automatic: No coding needed for basic controls.

Page-specific: Data stays on the same page.

Independent: No server resource usage (stored on client).

Limitations:

Increases page size: Every postback carries ViewState data.

Not secure by default: Can be tampered with (needs encryption if sensitive).

Page-specific only: Cannot share across pages.



Cookies

Cookies are **small text files** stored in the **client's browser** to retain **user-specific information** such as login credentials, preferences, or session identifiers. They are automatically **sent to the server with every HTTP request**, enabling personalized experiences across multiple pages or visits. Cookies can be **temporary (session cookies)**, which are deleted when the browser closes, or **persistent**, lasting for a set duration. **Size is limited (~4KB)**, making them suitable only for **lightweight data**, and while convenient, they are **less secure** if not encrypted.



Caching

Caching is a **performance optimization technique** used to **temporarily store frequently accessed data**—such as web pages, database results, or controls—either on the **client (browser cache)** or the **server (ASP.NET cache)**. This reduces page load time and server processing, improving the overall responsiveness of web applications. Unlike cookies, cached data is **not user-specific** and does **not travel with each request**. Caching in ASP.NET can be **page-level (output caching)**, **partial (control caching)**, or **data caching**, with configurable expiration policies for efficiency.



Cookies

Cookies are **small text files** ($\approx 4\text{KB}$) stored in the **browser** to **remember user-specific data** like login IDs or preferences. They are sent with every request to the server.

Types: Session (temporary) & Persistent (long-term).

Pros: Retains data between visits, simple to use.

Cons: Less secure unless encrypted, small size limit.



Caching

Caching is a **performance technique** that **temporarily stores frequently used data** (pages, controls, or database results) on the **server or browser to speed up responses**.

Types: Output caching (full page), partial caching (controls), data caching (objects).

Benefit: Faster page loads, reduced server load.

Key point: Not user-specific, used for shared performance gains.

Session vs Cookies

Session:

Server-side, user-specific, **secure**.

Used for sensitive or temporary data (login info, carts).

Cookies:

- **Client-side**, lightweight, **less secure**.
- Used for persistent preferences (theme, language).

Use **Sessions** for secure data; **Cookies** for small, non-sensitive preferences.

Session vs Cookies vs cache

Feature	Cookies	Session	Cache
Storage	Client (Browser)	Server	Server/Client
Purpose	Store user prefs	Store user session	Improve performance
Size	~4KB	Server memory	Configurable
Security	Low (unless encrypted)	High	Medium
Lifetime	Set by dev (persistent or session)	Ends when browser closes / timeout	Set by cache policy
Use Case	Remember user choices	Shopping cart, login	Caching pages/data



Types of State in ASP.NET

Client-side state is maintained on the **user's browser** and includes mechanisms like **ViewState** (stores page-specific data in hidden fields), **Cookies** (small files for user preferences), **QueryString** (data passed in the URL), and **Control State** (essential control information). These are lightweight but less secure.

Server-side state is stored on the **web server** and includes **Session State** (user-specific data for the current session), **Application State** (global data shared by all users), and **Cache** (temporary data storage for performance). These are more secure and persistent but consume server resources.

In short:

Client-side = Fast, lightweight, but less secure. **Server-side** = Secure, scalable, but uses server memory.



URL Encoding

URL Encoding converts **special characters** in a URL (like spaces, &, =) into a **safe format** using percent codes (e.g., space = %20). This is essential because URLs can only contain limited characters. In ASP.NET, it ensures **query string data** or user input doesn't break the URL and prevents errors or security issues.

```
string encoded = Server.UrlEncode("Hello World!");
```

Use: Passing data in query strings, handling user input, and ensuring safe, valid URLs.

Query Strings

A **Query String** is a way to **pass data between pages** through the URL using ?key=value format. Multiple values are separated by &.

```
https://mysite.com/Product.aspx?id=10&name=Book
```

```
string id = Request.QueryString["id"];
```

Use: Passing data in query strings, handling user input, and ensuring safe, valid URLs.



Query Strings

Pass a product ID and name between pages.

<https://site.com/Product.aspx?id=25&name=Laptop>

```
string id = Request.QueryString["id"];  
string name = Request.QueryString["name"];  
lblOutput.Text = $"Product: {name} (ID: {id})";
```

Use: Passing data in query strings, handling user input, and ensuring safe, valid URLs.

URL Encoding vs Query Strings

Feature	Query Strings	URL Encoding
Purpose	Passes data between pages in the URL	Converts special characters to a safe URL format
Format	?key=value&key2=value2	%20 for space, %26 for &, etc.
Visibility	Visible in the browser address bar	Encoded to avoid breaking URLs
Use Case	Passing IDs, search terms, filters	Ensuring safe transmission of query data
Security	Less secure (can be modified easily)	Adds safety for special characters
ASP.NET Method	Request.QueryString["key"]	Server.UrlEncode() / UrlDecode()

Master Pages

Master Pages allow you to create a **consistent layout** across multiple pages in a web application. A Master Page (.master) defines common sections like headers, footers, or navigation, using `<asp:ContentPlaceholder>` where child pages can inject their content.



Master Pages

Content Pages then use `<asp:Content>` to fill these placeholders, inheriting the master's design. For larger applications, **nested master pages** can be used, where a child master page inherits from a parent for multi-level layouts (e.g., different layouts for admin vs. public pages). You can also access controls in a master page from a content page using `Master.FindControl("ControlID")`.

```
<asp:ContentPlaceHolder ID="MainContent"runat="server"></asp:ContentPlaceHolder>
```

Themes in ASP.NET

Themes control the **look and feel** of pages and controls. They are stored in the App_Themes folder and can include **.skin files** (for control styles) and **CSS** for advanced styling. A skin file defines default properties for controls (e.g., button colors), ensuring a consistent design. Themes can be applied per-page using `<%@ Page Theme="BlueTheme" %>` or globally in Web.config with `<pages theme="BlueTheme" />`.

```
<asp:Button runat="server" BackColor="Blue" ForeColor="White" />
```



ASP.NET Navigation Controls

ASP.NET provides **navigation controls** to create structured, user-friendly site navigation, usually linked with a **Web.sitemap** file that defines the website hierarchy in XML.

- ☐ **Web.sitemap**: Defines the site structure (pages and their hierarchy).
- ☐ **Menu Control**: Displays a **horizontal/vertical menu** for navigating pages; supports multi-level items.
- ☐ **TreeView Control**: Shows a **collapsible tree structure** for nested sections (useful for dashboards or category navigation).
- ☐ **SiteMapPath (Breadcrumb)**: Displays the **current page path** (e.g., Home > Products > Details), improving user orientation.



Navigation in ASP.NET

ASP.NET provides navigation using **Web.sitemap**, which defines the site's hierarchy in XML. Navigation controls like **Menu** (hierarchical menus), **TreeView** (collapsible site structure), and **SiteMapPath** (breadcrumbs showing the current page path) bind to the sitemap for dynamic navigation.

```
<asp:Menu ID="Menu1" runat="server" DataSourceID="SiteMapDataSource1" />  
<asp:SiteMapDataSource ID="SiteMapDataSource1" runat="server" />
```